

BUILDER DESIGN PATTERN

- ① DEFINITION
- ② WHERE TO USE ?
- ③ PROBLEM STATEMENT
- ④ CODE

① DEFINITION

- ① Creational design pattern.
- ② It is used to construct complex objects step-by-step, especially when the object has many optional or configurable parameters.

② WHERE TO USE ?

- Too many parameters in a class.
- Too many optional parameters in a class.
- **Immutability**:- When you want to create an object that can't be changed after creation, but the creation process is complex.
- If you want to perform some operation before creating the object.

③ PROBLEM STATEMENT

① Employee class with lots of parameters.
→ name
→ email
→ address
→ salary
→ skill

② Employee class should be immutable.

③ Validate parameters before creating Employee class object.

validate:-

salary > 2000 And
skill == "java" And
name length minimum 3 char.

SOLUTION-1

```
package com.dp.builder.solution1;

public class Employee {
    private String name;
    private String email;
    private String address;
    private int salary;
    private String skill;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public int getSalary() {
        return salary;
    }

    public void setSalary(int salary) {
        this.salary = salary;
    }

    public String getSkill() {
        return skill;
    }

    public void setSkill(String skill) {
        this.skill = skill;
    }
}
```

```
package com.dp.builder.solution1;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Employee e1 = new Employee();
        e1.setName(name: "LoopKaka");
        e1.setEmail(email: "abc@gmail.com");
    }
}
```

PROBLEM WITH SOLUTION-1

This class is not immutable because it exposes public setter methods. After an Employee object is created, its field values can still be changed using methods like setName(), setEmail(), setSalary(), etc.

SOLUTION-2

```
package com.dp.builder.solution2;

public class Employee {
    private final String name;
    private final String email;
    private final String address;
    private final int salary;
    private final String skill;

    public Employee(String name, String email, String address, int salary, String skill) {
        this.name = name;
        this.email = email;
        this.address = address;
        this.salary = salary;
        this.skill = skill;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getAddress() {
        return address;
    }

    public int getSalary() {
        return salary;
    }

    public String getSkill() {
        return skill;
    }
}
```

```
package com.dp.builder.solution2;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Employee emp1 = new Employee(
            name: "LoopKaka",
            email: "abc@gmail.com",
            address: "India",
            salary: 0,
            skill: null);
    }
}
```

PROBLEMS WITH SOLUTION-2

- ① If a constructor has many fields, it becomes difficult to remember the correct sequence of parameters. The caller must pass values in the exact order expected by the constructor.
- ② If fields like salary or skill are optional, the caller still has to pass placeholder values such as 0 or null.

- ③ Optional parameters are difficult to maintain with a constructor-based approach. If more optional fields are added later, the constructor becomes longer and harder to use correctly.

SOLUTION-3

```
public Employee(String name, String email, String address) {  
    this.name = name;  
    this.email = email;  
    this.address = address;  
}  
  
public Employee(String name, String email, int salary) {  
    this.name = name;  
    this.email = email;  
    this.salary = salary;  
}  
  
public Employee(String name, String email, String skill) {  
    this.name = name;  
    this.email = email;  
    this.skill = skill;  
}
```

PROBLEM WITH SOLUTION-3

- ① Constructor overloading becomes confusing when many fields have the same type. We can also face compilation errors if multiple constructors have the same signature.

* Example: `Employee(String name, String email, String address)` and `Employee(String name, String email, String skill)` both become `Employee(String, String, String)` for Java.

- ② We can end up creating too many parameterized constructors. In the worst case, if we try to support every possible combination of optional fields, the number of constructors can grow close to 2^n , where n is the number of optional parameters.

- ③ Optional parameters are still difficult to manage. Constructor overloading does not handle all combinations cleanly, especially when multiple optional fields have the same data type.

SOLUTION - 4 (BUILDER DESIGN PATTERN)

PRACTICAL